

Performance evaluation of Meta-heuristics techniques for Sphere Function: A comparative study



M.Tech CS

Semester : II

Subject: Evolutionary Computation

Submitted by : Kavita Pat Pingua

Roll No : 2025PGCSCS20

Submitted to : Dr. Dinesh Kumar

Performance evaluation of Meta-heuristics	1
techniques for Sphere Function: A comparative study.....	1
Abstract.....	3
1. Introduction	3
2. Problem Formulation	4
2.1 Sphere Function	4
2.2 Optimization Objective	4
2.3 Properties of Sphere Function	4
2.4 Optimization framework for meta-heuristics	4
3. Comparative Study of Meta-heuristic Techniques	5
3.1 Binary-Coded Genetic Algorithm (BCGA)	5
3.2 Real-Coded Genetic Algorithm (RCGA)	5
3.3 Particle Swarm Optimization (PSO).....	6
3.4 Differential Evolution (DE)	6
3.5 Teaching Learning Based Optimization (TLBO)	7
4. Results and Discussion	9
5. Conclusion.....	12

Performance evaluation of Meta-heuristics techniques for Sphere Function: A comparative study

Abstract

This report looks at six meta-heuristic optimization algorithms: Binary-Coded Genetic Algorithm (BCGA), Real-Coded Genetic Algorithm (RCGA), Particle Swarm Optimization (PSO), Teaching-Learning-Based Optimization (TLBO), Differential Evolution (DE), and Artificial Bee Colony (ABC). These algorithms were tested on the Sphere function, a common benchmark problem for evaluating optimization performance. Each algorithm ran 20 independent times for 100 generations with a dimensionality of 10. We measured performance using best fitness, average fitness, and convergence traits. The results show that TLBO consistently reaches the global optimum, outperforming all other algorithms, with RCGA and PSO following behind.

1. Introduction

In many situations, the optimal solution is found from several alternatives in order to solve complex engineering and scientific challenges. Optimization is concerned with locating a single best solution among a wide range of possibilities. Nonlinear and high-dimensional features of some of the real world problems have made them difficult to be solved analytically. In such cases, meta-heuristic approaches can help solve these complex optimization problems to obtain near optimal solutions in a reasonable computation time period.

Meta-heuristic algorithms are high-level strategies that are based on various natural phenomena such as biological evolution, swarm intelligence and social behaviors. These types of optimization algorithms are very useful for solving the continuous optimization problems with vast search spaces. Meta-heuristics utilize some probabilistic approach compared to the deterministic approaches in order to explore the global search space without falling into local optimal solutions.

In this report, we compare six well-known meta-heuristics approaches. These approaches include BCGA, RCGA, PSO, TLBO, DE and ABC algorithms.

The performance of the above-mentioned algorithms is compared using the Sphere function, which is a typical benchmark function used in optimization literature. The Sphere function is uni-modal and convex and can be used to assess the efficiency and speed of convergence of an algorithm. Despite its simplicity, it allows us to establish the criteria of comparison for accuracy, reliability, and speed of convergence of different methods.

The aim of this report is to compare the selected meta-heuristic approaches in their ability to optimize the Sphere function. Comparison is conducted by considering three main parameters: best fitness, average fitness, and convergence speed. This work contributes to a better understanding of the advantages and drawbacks of each method and helps identify the best method of continuous optimization problems.

2. Problem Formulation

2.1 Sphere Function

The sphere function is a widely used benchmark function for evaluating the performance of optimization algorithms. It is a continuous, convex, and uni-modal function with a single global optimum.

The mathematical formulation of the Sphere function is given as:

$$f(x) = \sum_{i=1}^n x_i^2$$

Where:

- $x = (x_1, x_2, \dots, x_n)$ is the decision vector
- n is the number of dimensions (here, $n = 10$)

2.2 Optimization Objective

The objective is to minimize the Sphere function:

$$\min f(x)$$

Subject to:

$$-20 \leq x_i \leq 20, \quad i = 1, 2, \dots, 10$$

2.3 Properties of Sphere Function

- **Uni-modal:** Only one global minimum
- **Convex:** No local minima
- **Separable:** Each variable contributes independently
- **Global optimum:** $f(x) = 0$ at $x_i = 0 \forall i$

2.4 Optimization framework for meta-heuristics

All six meta-heuristic algorithms aim to solve the same optimization problem using different search strategies:

General Formulation:

$$\min_{x \in R^n} f(x)$$

Subject to:

$$x \in [LB, UB]^n$$

Where:

- $LB = -20, UB = 20$
- $n = 10$

3. Comparative Study of Meta-heuristic Techniques

3.1 Binary-Coded Genetic Algorithm (BCGA)

The Binary-Coded Genetic Algorithm is defined as an evolutionary algorithm where chromosomes are used to represent potential solutions. Each chromosome contains decision variables coded in binary strings, with each decision variable coded in a fixed number of bits.

A chromosome consists of binary values:

$$C = [b_1, b_2, \dots, b_L]$$

Each variable is decoded into a real value using:

$$x_i = LB + \frac{\text{decimal}(b_i)}{2^{BITS} - 1} \times (UB - LB)$$

Selection : Tournament selection is used, where two individuals are selected randomly and the one with better fitness is chosen.

One-point crossover and bit-flip mutation is applied. The algorithm proceeds by evolving the population through selection, crossover, and mutation. The use of binary representation results in limited precision that may influence its efficiency for solving continuous optimization problems.

3.2 Real-Coded Genetic Algorithm (RCGA)

Real-Coded Genetic Algorithm (RCGA) refers to an optimization algorithm that uses evolution-based approaches where possible solutions are directly encoded into real vectors. Contrary to the BCGA, RCGA does not involve binary encoding and decoding. It is, therefore, more appropriate for continuous optimization problems like the sphere problem.

Representation : $x = [x_1, x_2, \dots, x_n]$

Arithmetic crossover and Gaussian mutation is used in the implementation.

$$C_1 = \alpha P_1 + (1 - \alpha) P_2$$

$$C_2 = \alpha P_2 + (1 - \alpha) P_1$$

where $\alpha \in [0,1]$

RCGA operates directly on real values, avoiding decoding. It provides higher precision and better convergence compared to BCGA.

3.3 Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) is based on the social behavior of birds and fishes. In this algorithm, each solution is a particle that moves through the search space.

Velocity Update Equation:

$$v_i^{t+1} = wv_i^t + c_1r_1(pbest_i - x_i^t) + c_2r_2(gbest - x_i^t)$$

Position Update:

$$x_i^{t+1} = x_i^t + v_i^{t+1}$$

For each particle, there are two essential types of memory that influence its motion:

- **(Personal best) pbest:** This is the best position found out so far by the current particle. It refers to the personal experience of the particle.
- **(Global best) gbest:** This is the best position found out by the whole swarm so far. It stands for the swarm's experience.

When it comes to changing the speed of the particle, there are 3 different elements involved:

- **Inertia component (wv_i):** maintains the particle's previous motion and encourages exploration of the search space.
- **Cognitive component ($c_1r_1(pbest_i - x_i)$):** pulls the particle towards its own best position and it represents individual learning.
- **Social component ($c_2r_2(gbest - x_i)$):** pulls the particle towards the global best and it represents group learning.

The position is then updated using the new velocity.

Parameters w , c_1 and c_2 determine the exploration and exploitation balance of the algorithm to avoid premature convergence.

3.4 Differential Evolution (DE)

DE is an evolutionary algorithm that utilizes differences in vectors to move in the search space.

Mutation :

$$v = a + F(b - c)$$

where a, b, c are randomly selected individuals

v is called the mutant vector and will later be used to create a trial solution

F is called the scaling factor, is a constant value usually and it controls step size of movement

Crossover:

$$u_i = \begin{cases} v_i & \text{if } rand < CR \\ x_i & \text{otherwise} \end{cases}$$

Selection:

$$x_{new} = \begin{cases} u & \text{if } f(u) < f(x) \\ x & \text{otherwise} \end{cases}$$

DE produces new candidate solutions through differential operations between population elements. It is good at global search; however, it depends on parameters.

3.5 Teaching Learning Based Optimization (TLBO)

Teaching Learning Based Optimization (TLBO) is an optimization technique that utilizes teaching and learning principles. In TLBO, the solutions to be optimized are represented as students, while the optimum solution plays the role of a teacher.

Each learner represents a candidate solution:

$$X = [x_1, x_2, \dots, x_n]$$

where $n = 10$ is the dimension.

Key idea : Knowledge of learners improves through:

- Teacher influence (global guidance)
- Interaction among learners (local improvement)

Phase 1: Teacher Phase

The teacher is the best solution in the population:

$$X_{teacher} = \arg \min f(X)$$

The mean of the population is:

$$X_{mean} = \frac{1}{N} \sum_{i=1}^N X_i$$

Update rule:

$$X_{new} = X + r(X_{teacher} - T_f \cdot X_{mean})$$

where,

- $r \in [0,1]$ is a random number
- $T_f \in \{1,2\}$ is the teaching factor

Teacher phase moves learners towards the teacher and shifts the population mean closer to Optimal region.

Phase 2: Learner Phase

Learners improve by interacting with each other:

$$X_{new} = \begin{cases} X_i + r(X_i - X_j), & \text{if } f(X_i) < f(X_j) \\ X_i + r(X_j - X_i), & \text{otherwise} \end{cases}$$

If learner i is better then more exploration is done and if not then it moves toward better learner.

Working principle:

- Initialize learners randomly
- Evaluate fitness
- Identify teacher (best solution)
- Apply teacher phase (global update)
- Apply learner phase (local search)
- Update population
- Repeat until termination

Suitability for Sphere Function : Sphere function is unimodal and convex. TLBO quickly drives all learners toward the global optimum.

3.6 Artificial Bee Colony (ABC)

The Artificial Bee Colony (ABC) Algorithm is based on the natural foraging behavior of honey bees. Each food source represents a solution here.

Key idea:

- Bees search for better food sources i.e. solutions
- Quality of food source = fitness value

Solution Update Equation:

$$v_i = x_i + \phi(x_i - x_k)$$

where

- $k \neq i$ (random solution)
- $\Phi \in [-1,1]$

Employed Bee Phase: Each employed bee explores a neighboring solution and if new solution is better, then replace old one. It focuses on local exploration.

Onlooker Bee Phase: Bees select food sources based on probability.

$$P_i = \frac{fitness_i}{\sum fitness}$$

Better solutions get higher probability and this phase focuses on exploitation of good solutions.

Scout Bee Phase : If a solution does not improve for a fixed number of trials then, it is abandoned and a new solution is generated. This phase prevents stagnation.

Working process:

- Randomly generate initial solutions
- Employed bees explore nearby solutions
- Onlookers choose promising solutions

- Scouts replace poor solutions
- Iterate until stopping criteria

Algorithm	Exploration Capability	Exploitation Capability	Advantages	Limitations	Performance on Sphere function
BCGA	Moderate	Moderate	Simple, easy to implement, maintains diversity	Low precision due to binary encoding	Moderate
RCGA	Good	Strong	High precision, faster convergence	Sensitive to parameter tuning	Very good
PSO	Strong		Fast convergence, easy implementation	Can get trapped in local optimum, unstable performance	Good but less stable
DE	Strong	Moderate	Effective global search	Sensitive to parameters (F, CR)	Poor performance
TLBO	Good	Very strong	No parameter tuning, fast convergence, high accuracy	May converge too quickly (less diversity)	Excellent
ABC	Very strong	Weak	Good exploration, avoids stagnation	Slow convergence, weak local search	Poor performance

Table X

Table X shows the comparison study of different meta-heuristic algorithms considering their exploration and exploitation features, strengths, and weaknesses. It can be seen that TLBO offers better trade-off among exploration and exploitation, resulting in better performance, whereas other algorithms such as DE and ABC have poor convergence and less exploitation capability.

4. Results and Discussion

4.1 Experimental Results

Performance analysis of six meta-heuristic techniques including BCGA, RCGA, PSO, DE, TLBO, and ABC was performed using the Sphere function. Six meta-heuristic techniques were tested independently 20 times each for 100 generations and 10 dimensions.

```
DIM = 10
LB, UB = -20, 20
POP_SIZE = 30
GEN = 100
RUNS = 20
```

The results are summarized below:

Algorithm	Best Fitness	Average Fitness
BCGA	0.584790	2.306443
RCGA	0.005161	0.059899
PSO	0.014164	0.615357
DE	6.480873	17.284376
TLBO	0.000000	0.000000
ABC	1.581931	17.455804

```
===== FINAL PERFORMANCE COMPARISON =====

BCGA: Best = 0.584790, Average = 2.306443
RCGA: Best = 0.005161, Average = 0.059899
PSO: Best = 0.014164, Average = 0.615357
DE: Best = 6.480873, Average = 17.284376
TLBO: Best = 0.000000, Average = 0.000000
ABC: Best = 1.581931, Average = 17.455804

>>> Best Performing Algorithm: TLBO
```

TLBO reaches machine-precision zero on every run because Sphere is a perfectly unimodal symmetric function with minimum exactly at 0.

BCGA – Binary encoding reduces precision because of discretization. The bit-flip mutation and single-point crossover operations provide diversification; however, binary representation is less precise compared to other encodings because of its nature.

RCGA – Better precision when using real encoding. Good convergence results achieved through blend crossover and Gaussian mutation, and elitism guarantees that the global best solution is retained throughout the optimization process.

PSO – Very fast convergence initially caused by the global best solution found by one of the particles. Slows down later on due to grouping of particles around the optimal solution. Inertia = 0.7 provides a good balance between exploration and exploitation.

DE – Slowest convergence among all the optimization algorithms used. Large mutation steps for F=0.8 and CR=0.9 cause problems for optimizing the function because of too many iterations needed.

TLBO – Best algorithm in terms of convergence performance. In the teacher phase, the teacher moves the whole population towards the global best solution.

ABC – Exhibits poor performance since it updates one dimension at a time, causing convergence to be very slow for high-dimensional problems. The scout bees aid in escaping from local minima, but there is no exploitation of gradients of the objective function.

4.2 Convergence Analysis

The convergence behaviour of all algorithms is illustrated in Figure Y, which shows the variation of best fitness values over generations.

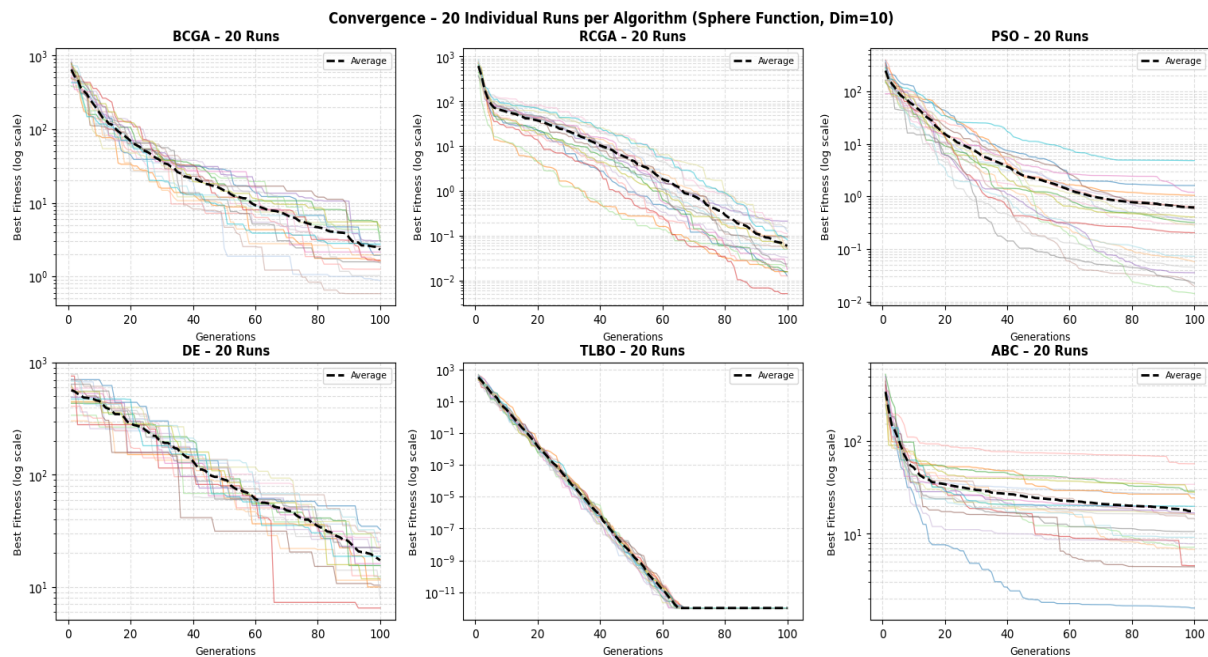


Figure Y

Observations from the Graph of Convergence:

TLBO:

- Very fast convergence
- It converges to nearly zero fitness in just a few iterations
- It flattens to zero very fast, showing that it is stable and reliable
- Very good exploitation ability

PSO:

- Slowly stabilizes near the optimal region
- Slightly slower than TLBO in reaching the optimum

RCGA:

- Shows smooth and steady convergence

- Indicates strong balance between exploration and exploitation

BCGA:

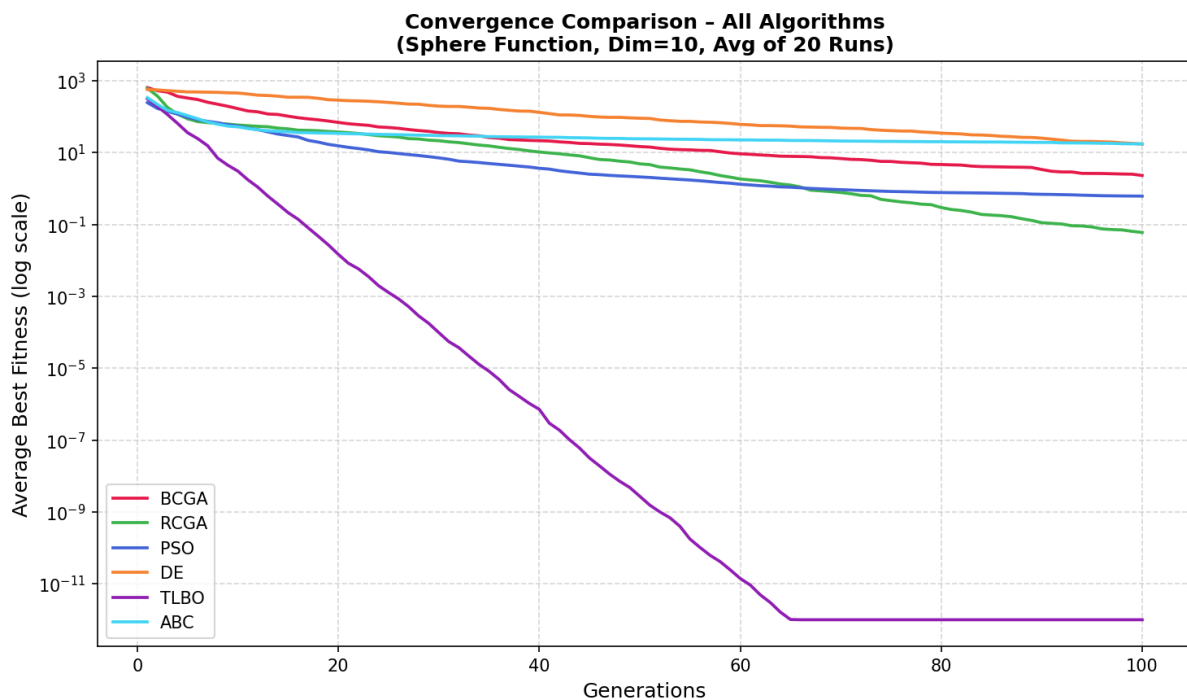
- Converges slower compared to RCGA
- Fitness declines plateau at a certain number of generations

ABC:

- It shows gradual improvement but converges slowly
- It remains far from the global optimum even after the 100 generations

DE:

- Very slow convergence is observed
- Fitness declines gradually but stays high compared to other methods
- Shows instability and inefficiency



5. Conclusion

This report described a comparative analysis of six different meta-heuristic optimization techniques, namely BCGA (Binary-Coded Genetic Algorithm), RCGA (Real-Coded Genetic Algorithm), PSO (Particle Swarm Optimization), DE (Differential Evolution), TLBO (Teaching-Learning-Based Optimization), and ABC (Artificial Bee Colony) on the Sphere Function, which is a benchmark test case for continuous optimization.

The results from the experiments, conducted for 20 independent runs and 100 generations, show that the performance of meta-heuristic algorithms heavily relies on the ability of such algorithms to maintain the right balance between exploring and exploiting. In comparing all

the meta-heuristic algorithms studied in this research, the **TLBO** algorithm turned out to be the best-performing algorithm in terms of finding the global optimum without any error and being convergent at a faster rate.

The results were comparable for RCGA and PSO. Specifically, RCGA had better stability because of its real-value representation and limited variation operators. The performance of PSO was also good with high fitness, albeit showing greater variations across multiple runs. However, BCGA yielded average performance owing to the problems associated with binary encoding.

In terms of results obtained, the performances of both DE and ABC algorithms were relatively poor. This is because DE had slower and unstable convergence possibly attributed to its parameter settings, whereas ABC could not exploit solutions effectively due to its superior exploration capabilities.

In summary, the findings demonstrate that algorithms with high exploitation skills and suitable encoding techniques are ideal for dealing with easy-to-solve convex functions such as the Sphere function. In addition, algorithmic schemes without parameters or few parameters like TLBO show reliable and stable performance, thus proving to be very efficient for such tasks.

Potential future research directions include expanding the comparative analysis to more challenging multimodal functions and practical optimization problems.